

Graph ds & algo.

-E.g.1. Roads/Rails that connect cities. Compute the shortest route between two cities.

-E.g.2. Internet & Social Network.

(Vertices = People
Edges = Pairs of people)



Qn: What's the degree of separation between two people in the world? Empirically = 6?

- Mathematical modeling of these problems is best done by:

Defn: Graph $G = (V, E)$ has V : vertices $= [n]$,
& edges $E \subseteq V \times V = [n] \times [n]$.

- E.g.1. Undirected G : $(x, y) \in E \iff (y, x) \in E$.

$G_1 = (V_1, E_1); V_1 = [4]$

$E_1 = \{ (1, 2), (2, 1), (1, 3), (3, 1), (3, 4), (4, 3) \}$



- E.g.2. Directed G :
 $E_2 = \{ (1, 2), (1, 3), (3, 4) \}$.

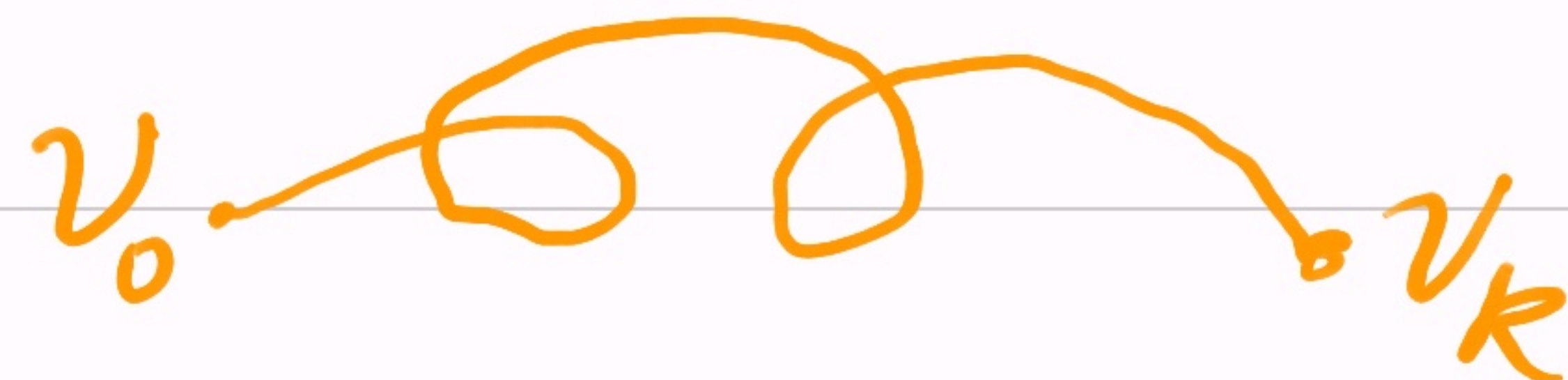


▷ $n = |V|$, $m = |E|$: directed $\Rightarrow 0 \leq m \leq n(n-1)$.
 $0 \leq \# \text{ undir. edges} \leq \binom{n}{2}$.
[we're discounting self-loops.]

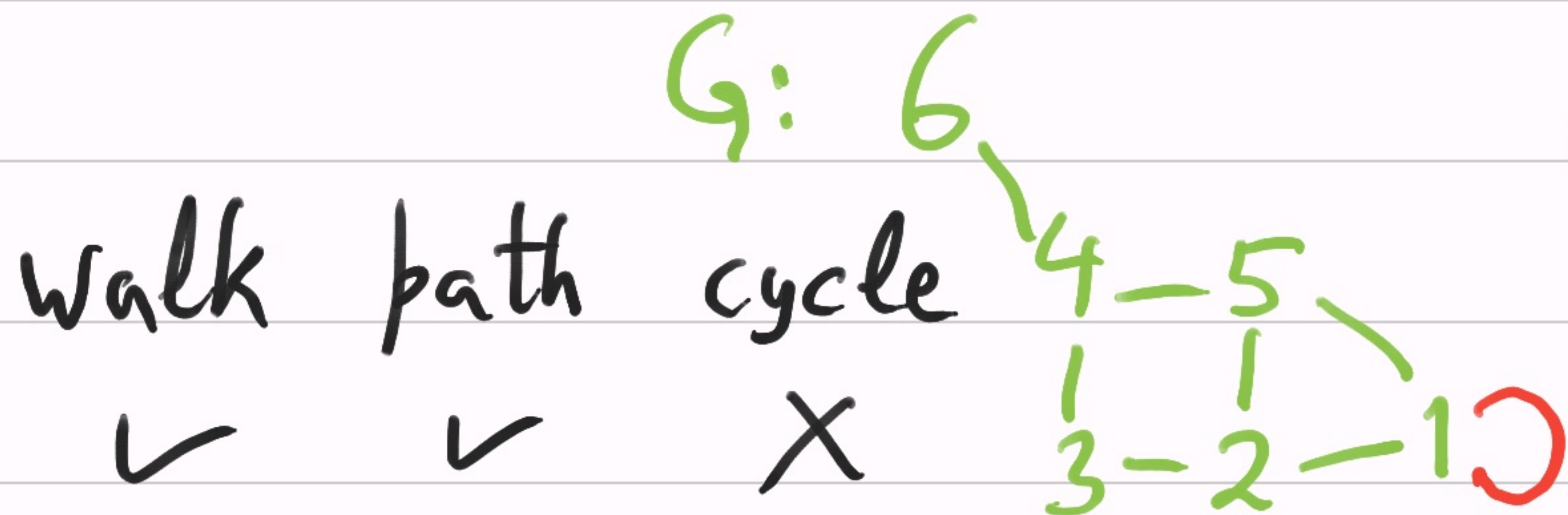
▷ Undirected graph is also directed, but not the converse.

- Defn: • List of vertices $L = \langle v_0, \dots, v_k \rangle$ is a walk if $(v_i, v_{i+1}) \in E$, for all $0 \leq i \leq k-1$.
- Walk L is a path if v_i 's are distinct.
 - Walk L is a cycle if v_i 's are distinct except

$v_0 = v_k$, (if $k=0$ then $(v_0, v_0) \in E$.)



E.g.

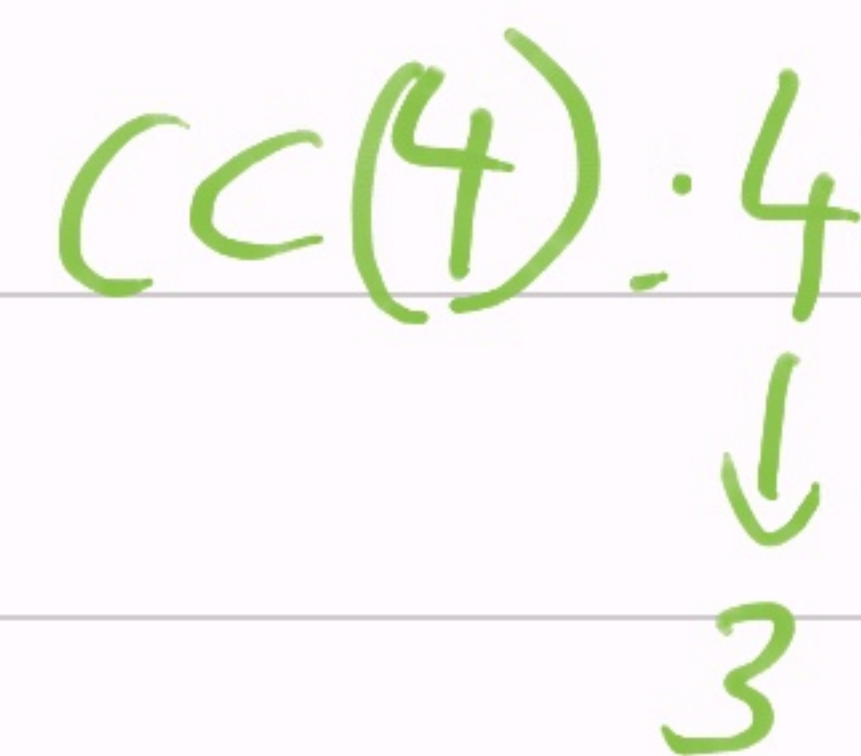
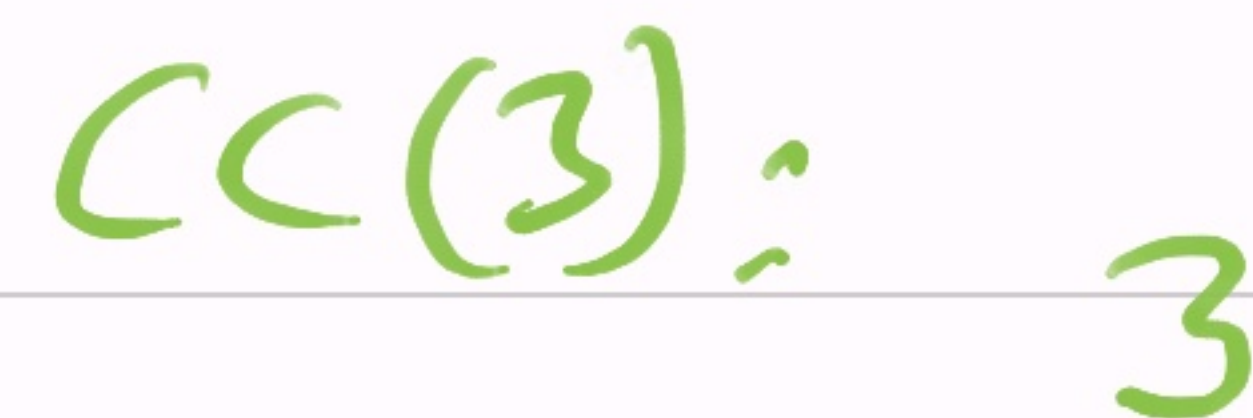
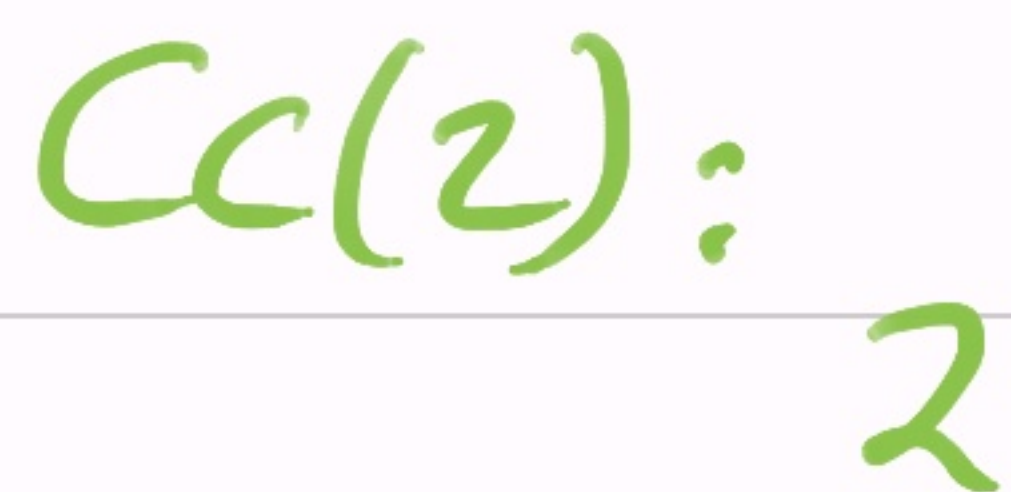
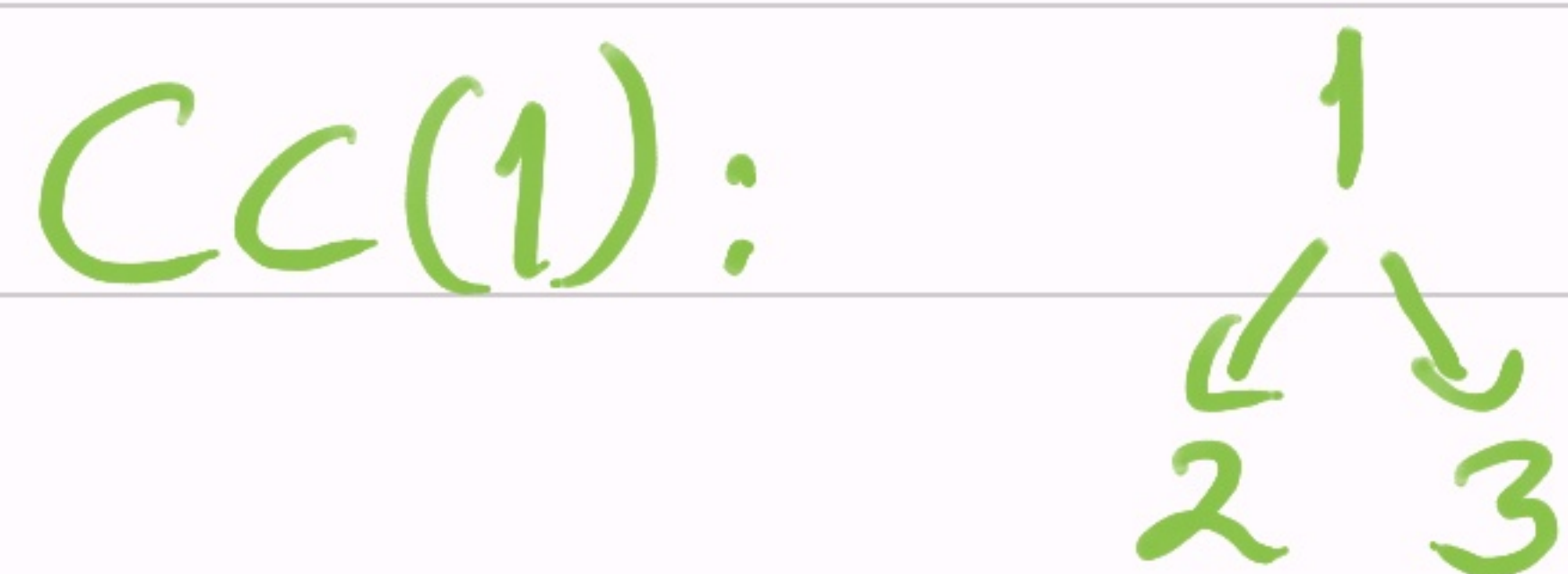
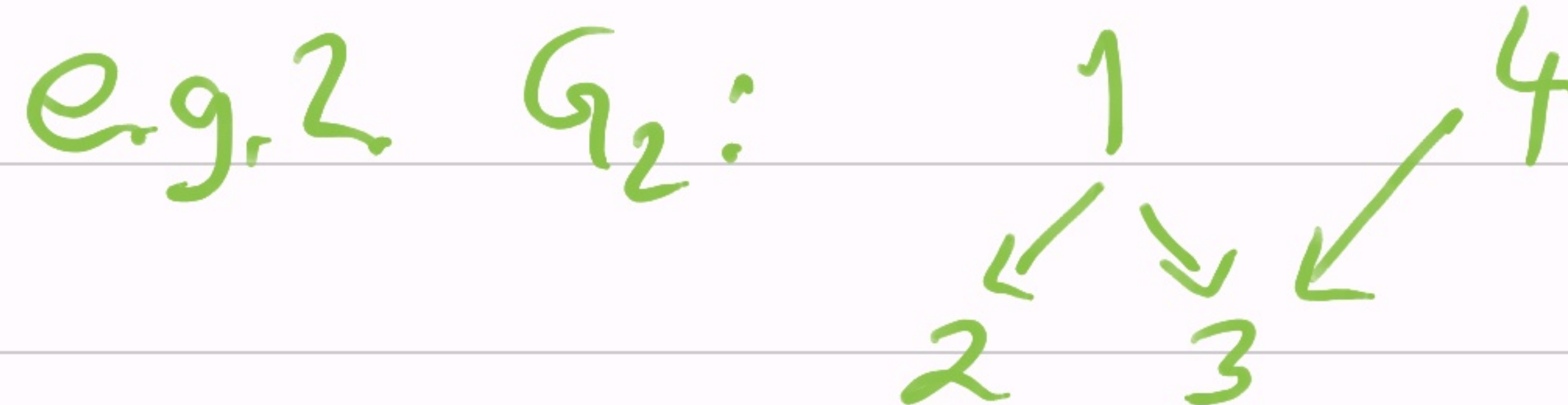
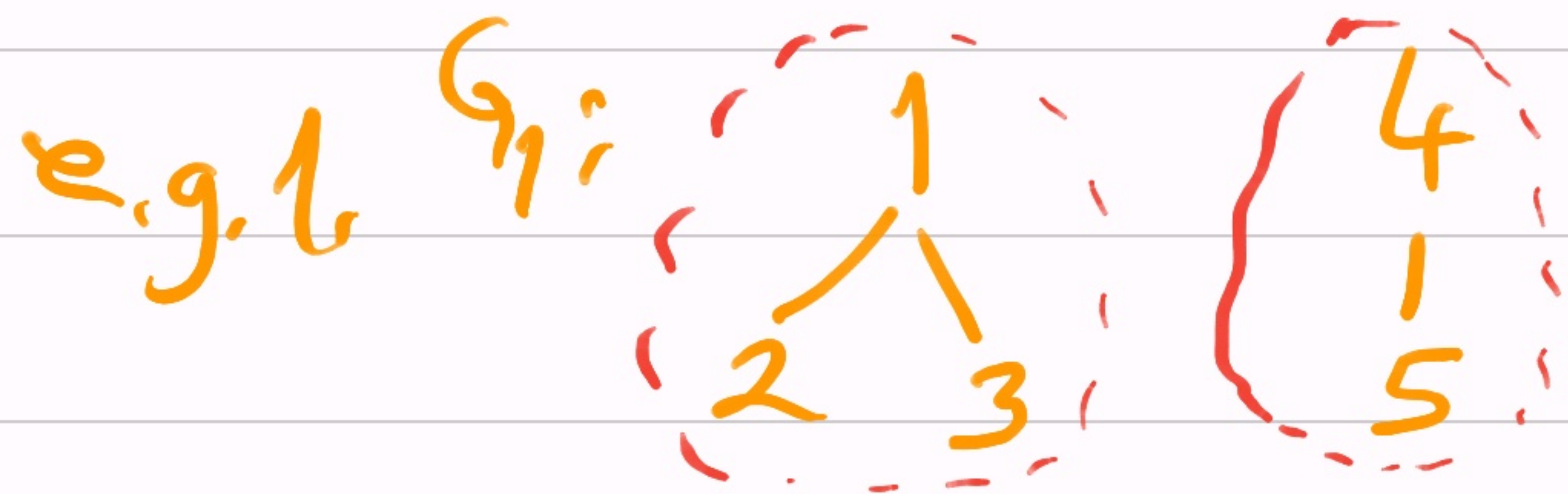


	walk	path	cycle
154 :	✓	✓	X
132 :	X	X	X
12523 :	✓	X	X
1251 :	✓	X	✓
6 :	✓	✓	X $[(6,6) \notin E]$
1 :	✓	✓	✓

- Vertices v_0, v_k are connected in G if $\exists$ path $\langle v_0, v_1, \dots, v_k \rangle$. (i.e. k-hops.)
- A maximal subset C of vertices V connected to v_0 is called a connected component (of v_0 in G). ↖ CC
- ▷ $\forall v \in V \setminus C$, v is disconnected from v_0 .

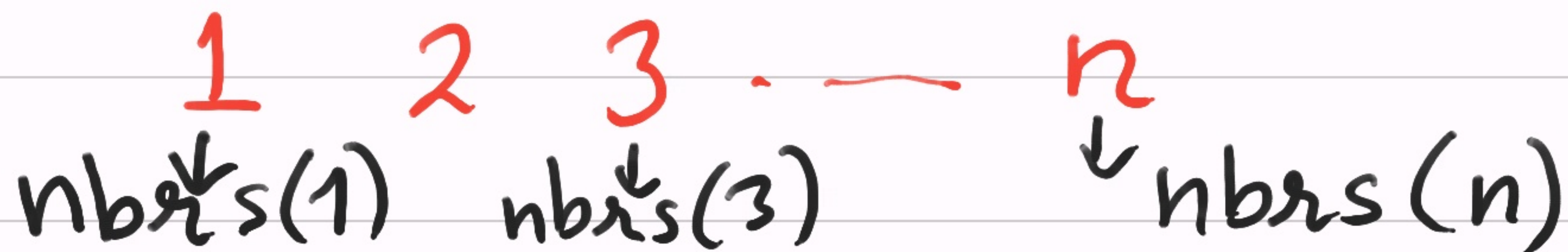
▷ In undirected G , if v has CC C & $v' \in C$ has CC C' , then $C = C'$.

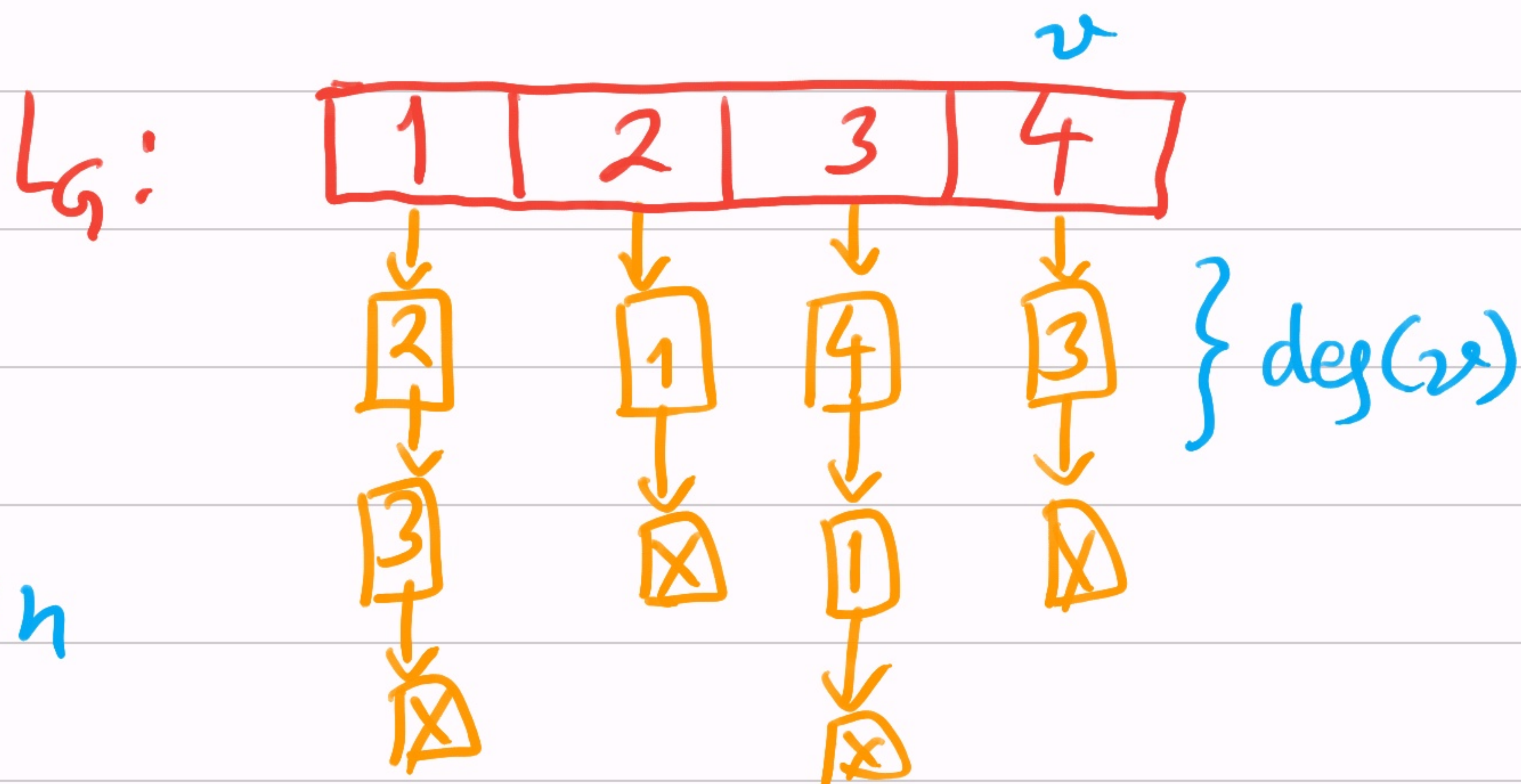
▷ In undirected G , there's a unique partition of V into (disjoint) CCs ($\leq n$ many).



Implementing a Graph (as a ds)

- Link-based: Adjacency list of G , $\underline{L}_G$:





$\triangleright L_G \leftrightarrow G$ are in 1-1 correspondence.

Pf: Exercise. $\square$

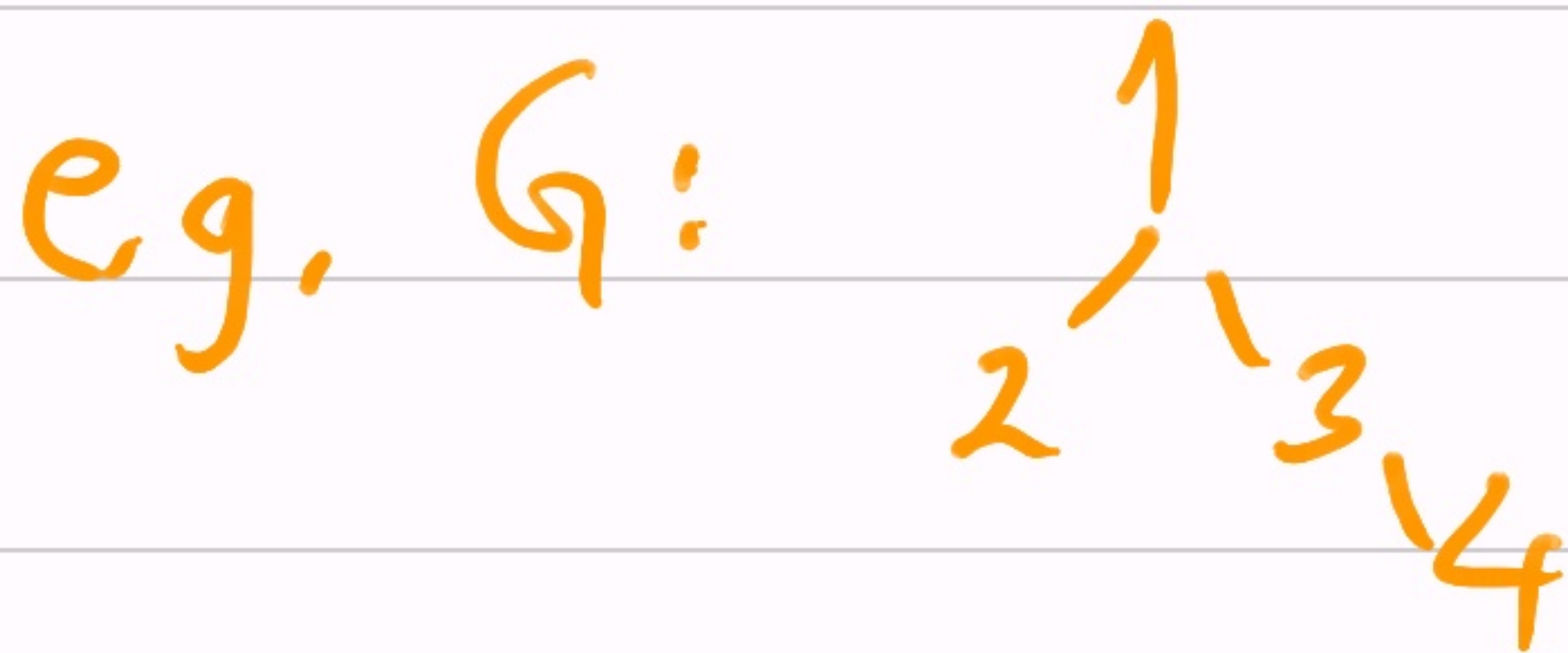
$\triangleright$ Given G , size of L_G is $\leq O(n+m)$.

$\triangleright \sum_{1 \leq i \leq n} |list(i)| \leq O(m)$. [For undirected G it's $= 2m$.]
 $\quad \quad \quad \text{"deg}(i)$

Pros: It optimizes the space. Finds all the neighbors fast.

Cons: Testing $(x, y) \in ? E$, takes $\deg(x) \leq O(n)$.

2D-array based implement: Adjacency matrix of G is $\underline{A_G}$, with (i, j) -entry $= \begin{cases} 1, & \text{if } (i, j) \in E \\ 0, & \text{else.} \end{cases}$



A_G :

	1	2	3	4
1	0	1	1	0
2	1	0	0	0
3	1	0	0	1
4	0	0	1	0

▷ size of A_G is $O(n^2)$. Pros: $(x, y) \in ? E$ takes $O(1)$ time.

Cons: Takes $O(n^2)$ even if $m \ll n$, i.e. it's bad for sparse graphs.

→ In real life, adjacency list is better!

- Defn:

- y is reachable from x if $\exists$ path: $x \leadsto y$.
- Traversal from x : We want to visit all vertices y reachable from x .

BFS (Breadth-first Search/Traversal)

- Defn: • Distance $d_G(x, y) :=$ (least #edges on a path $x \rightsquigarrow y$)
 $= |\text{shortest-path } x \rightsquigarrow y|$.

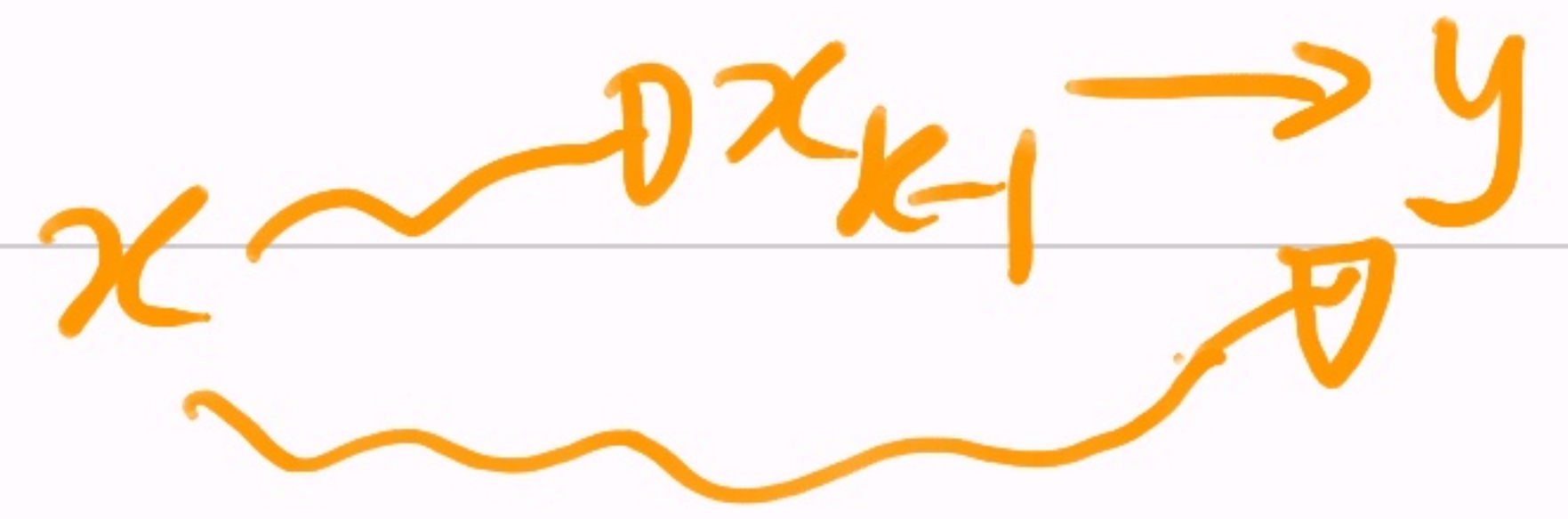
▷ $0 \leq d(x, y) \leq n-1$; or $d(x, y) = \infty$.

▷ $\langle x, x_1, \dots, x_{k-1}, y \rangle$ is shortest-path $x \rightsquigarrow y$,
then $d(x, y) = k$; $d(x, x_{k-1}) = k-1$.

Pf: • We've $d(x, x_{k-1}) \leq k-1$. If $d(x, x_{k-1}) \leq k-2$,
then we can use it to get

$$d(x, y) \leq d(x, x_{k-1}) + 1$$

[Triangle inequality] $\leq k-2+1 = k-1 < k,$



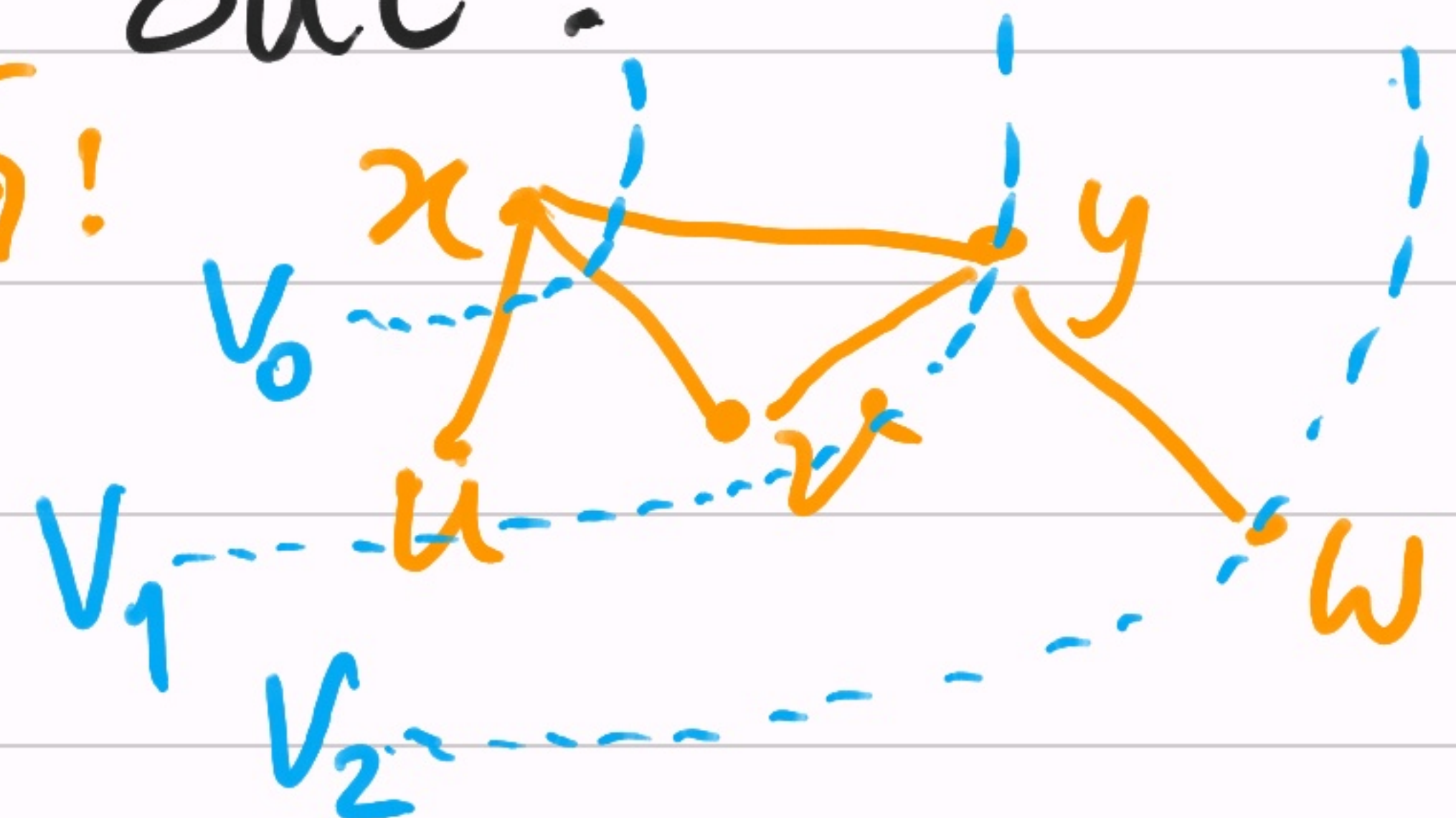
$\Rightarrow$ a contradiction! $\square$

$\Rightarrow$ Shortest paths have shortest subpaths!

— Idea of BFS from x : • Start a 'ripple' centered at x & expand it out.

• $V_0 = \{x\}$; $V_1 = \{u, v, y\}$; $V_2 = \{w\}$. $\hookrightarrow$

• Initialize $D(x) \leftarrow 0$;
other y 's: $D(y) \leftarrow \infty$;



- Compute $V_1 \leftarrow \text{nbr}(x);$
}

- For V_{i+1} : Put w in V_{i+1} iff
 $w \in \text{nbr}(V_i)$ AND $D(w) = \infty$,

→ This works out to a pseudocode...

$\text{BFS}(G, x) \{$

create EmptyQ (Q) ; enqueue (x) ;

$D[x] \leftarrow 0$; Other y 's: $D[y] \leftarrow \infty$;

while $(! \text{isEmpty}(Q)) \{$

$v \leftarrow \text{dequeue}(Q);$

```
For  $w \leftarrow \text{nbr}(v)$  :  
    if  $(D[w] = \infty)$  {  
         $D[w] \leftarrow D[v] + 1$ ;  
        enqueue( $w, Q$ );  
    }
```

} // end while

return $D[V]$; // all vertices V of G

} // end BFS

▷ Any vertex w is enqueued in Q at most once.

Pf: $w \neq x$ is enqueued $\Rightarrow D[w]$ is updated from

∞ to a finite value. Then, it's never changed again! $\square$

$$\triangleright \sum_{v \in V} \deg(v) \leq O(|E|) = O(m).$$

Pf: An edge is at most double-counted in the LHS. $\square$

$\Rightarrow$ It's slow when m is large (e.g. $m \geq n^2/2$).

Theorem [Moore '59]: BFS takes time $O(n+m)$.
It covers the $CC(x)$ & correctly computes

$$D[v] = d_G(x, v),$$

Pf: • Do induction on $d(x, w)$; and prove the
induction step: $x \sim w \rightarrow v$.

$$\Rightarrow d(x, v) \leq 1 + d(x, w) \quad \& \quad d(x, v) > d(x, w). \quad \square$$

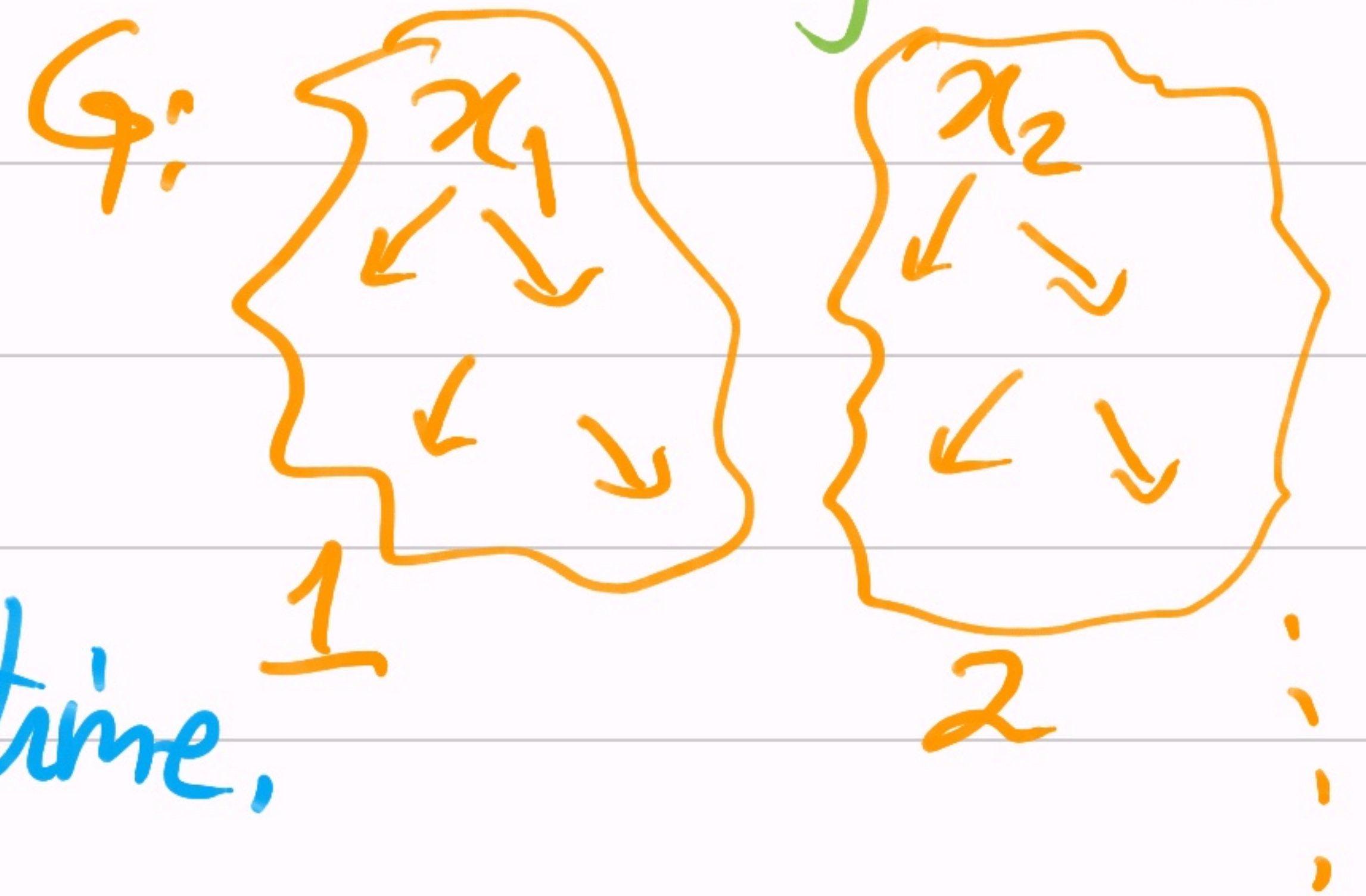
— BFS gives a new ds on the graph G .

Application 1: Answering " $u \sim v$?" in undirected
 G , in $O(1)$ time?

Idea: While doing BFS maintain a label that numbers the correct CC of the vertex.

▷ $u \sim v$ iff their labels are identical.

▷ Building the ds takes $O(n+m)$ time.

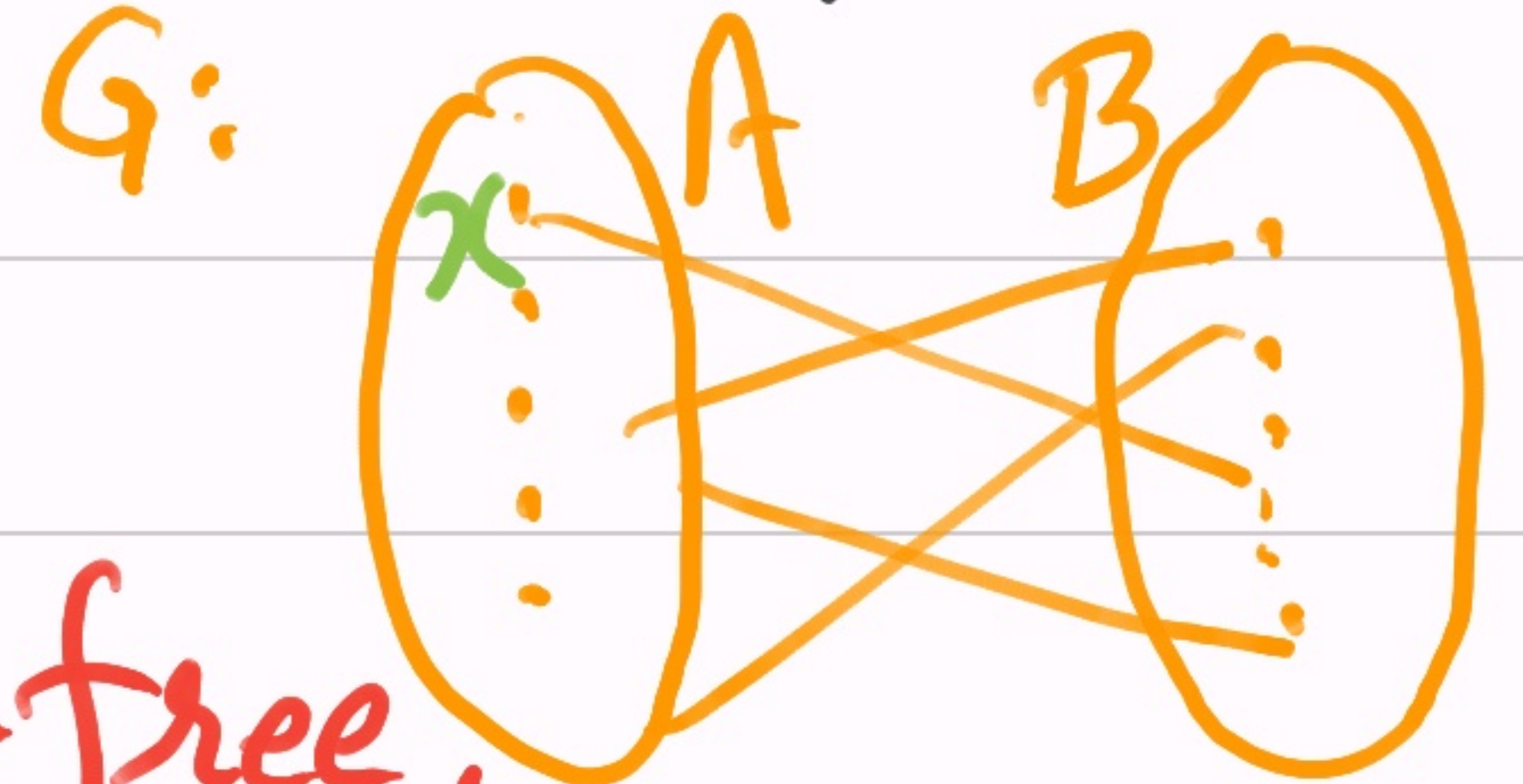


Application 2: Checking bipartiteness?

Defn: $G=(V,E)$ is bipartite if

$V=A \cup B$ is a partition &

$E \subseteq A \times B$. I.e. A, B are edge-free.



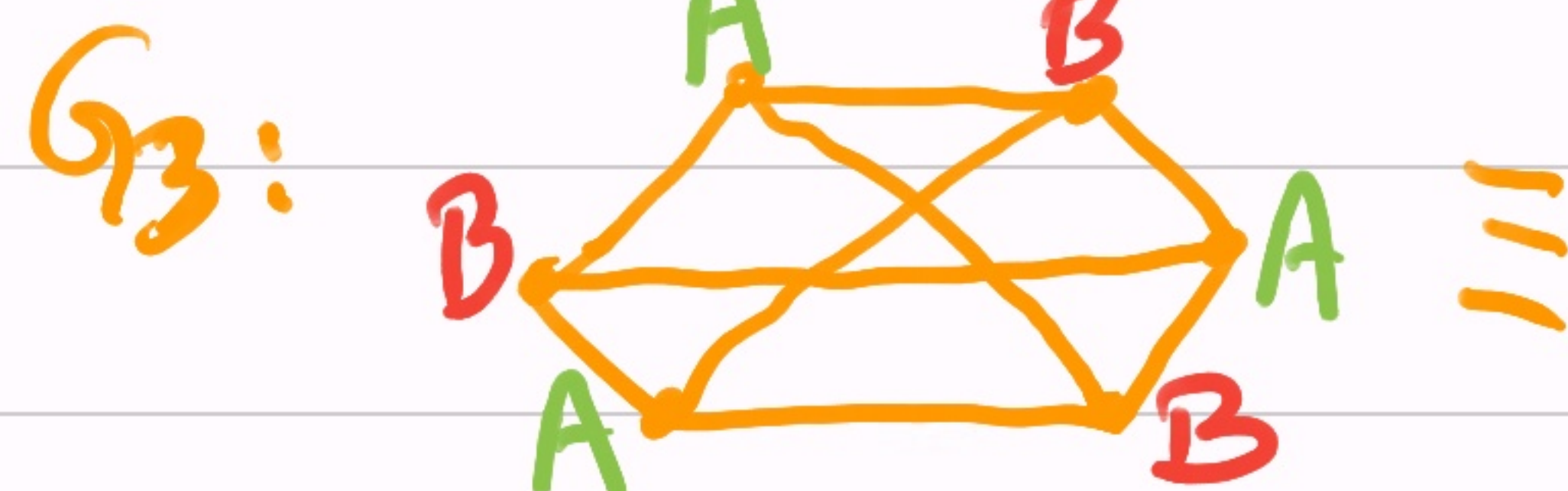
Idea 0: Go over all subsets $A \subseteq V$.
 $\Rightarrow 2^n$ -time. $\Rightarrow$ impossible time!

Input: G is a connected undirected graph,
given as adj. list.

Output: Bipartite rep. or say it doesn't exist.



Odd cycle is non-bip.



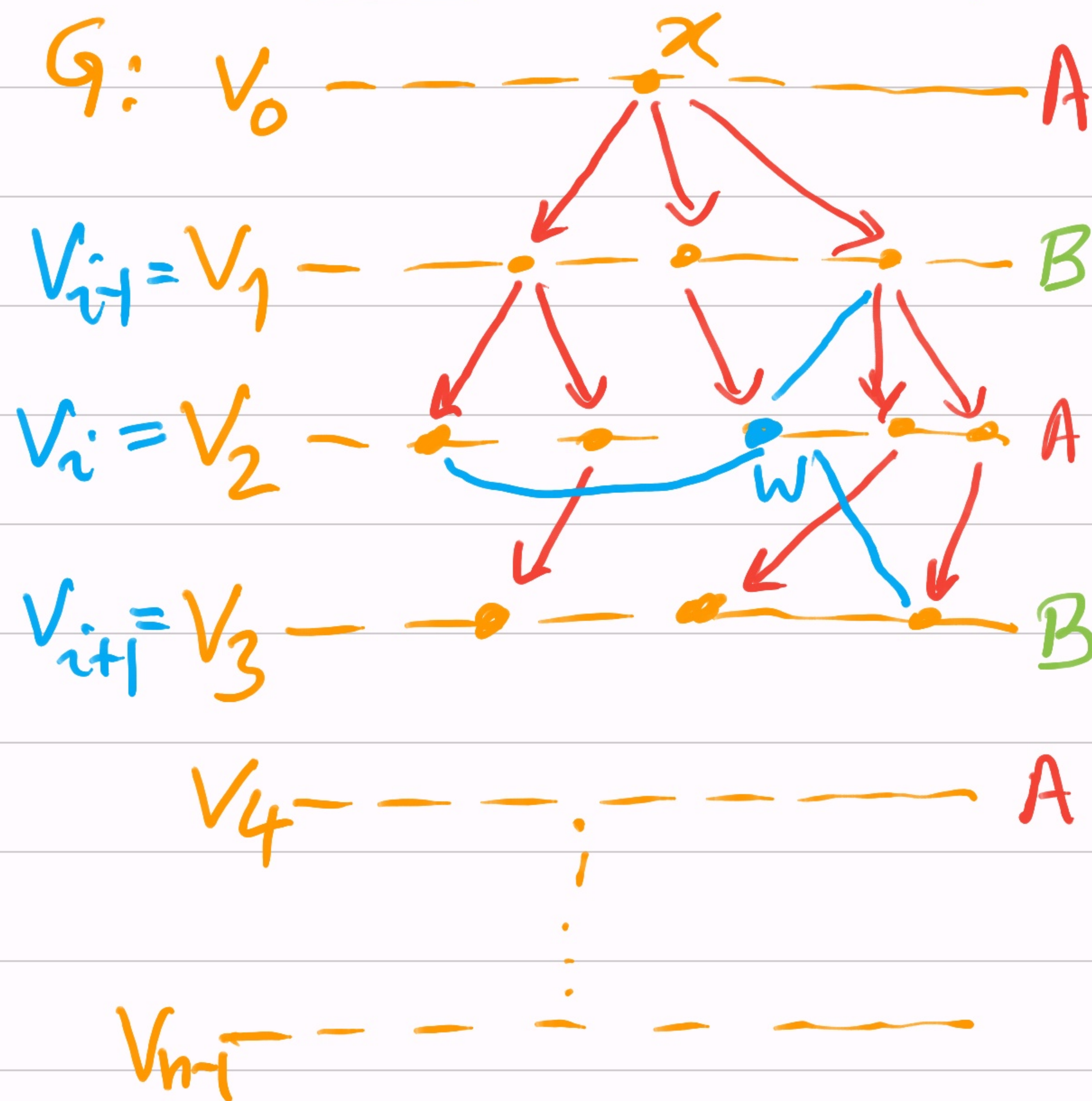
Even cycle is bip.

Idea 1: • BFS(G, x) gives a tree T , (=red edges)

▷ $w \in V_i$ can't have
edges to $V_{i-2}, V_{i-3}, \dots$
or to $V_{i+2}, V_{i+3}, \dots$

▷ non-tree-edges from
 V_i go to V_{i-1}, V_i, V_{i+1} ;
and nowhere else.

• $A :=$ even ripples, &
 $B :=$ odd " .



• If $\exists$ edge in $V_i \Rightarrow$ odd cycle exists using $\pi \Rightarrow G$ is non-bipartite.

• If no edge is there within $V_i, \forall i$, (there might be edges from V_i to $V_{i \pm 1}$), then
 $\Rightarrow V = A \cup B$ works
 $\Rightarrow G$ is bipartite!

$\triangleright e = (u, v)$ is blue inside $V_i \Leftrightarrow D[u] = D[v] = i$.

Theorem: Bipartite rep. can be found in $O(m+n)$ time.